

Learning vision-based agile flight via differentiable physics

Received: 15 July 2024

Accepted: 1 May 2025

Published online: 16 June 2025

 Check for updates

Yuang Zhang ^{1,3}, Yu Hu ^{1,3}, Yunlong Song ^{2,3}, Danping Zou ¹✉ & Weiyao Lin ¹✉

Autonomous aerial robot swarms promise transformative applications, from planetary exploration to search and rescue in complex environments. However, navigating these swarms efficiently in unknown and cluttered spaces without bulky sensors, heavy computation or constant communication between robots remains a major research problem. This paper introduces an end-to-end approach that combines deep learning with first-principles physics through differentiable simulation to enable autonomous navigation by several aerial robots through complex environments at high speed. Our approach directly optimizes a neural network control policy by backpropagating loss gradients through the robot simulation using a simple point-mass physics model. Despite this simplicity, our method excels in both multi-agent and single-agent applications. In multi-agent scenarios, our system demonstrates self-organized behaviour, which enables autonomous coordination without communication or centralized planning. In single-agent scenarios, our system achieved a 90% success rate in navigating through complex unknown environments and demonstrated enhanced robustness compared to previous state-of-the-art approaches. Our system can operate without state estimation and adapt to dynamic obstacles. In real-world forest environments, it navigates at speeds of up to 20 m s^{-1} , doubling the speed of previous imitation-learning-based solutions. Notably, all these capabilities are deployed on a budget-friendly US\$21 computer, which costs less than 5% of the GPU-equipped board used in existing systems.

Modern aerial robots are increasingly being called upon to execute complex, agile manoeuvres in dynamically changing and unknown environments, from search and rescue operations¹ to powerline inspections² or deliveries³. Beyond single-agent flight, a swarm-based vision-driven system represents a promising avenue for tackling those tasks more effectively. Such drones are required to operate with extremely limited sensory and computational resources onboard. The key to mastering vision-based flight is the tight integration of efficient, robust perception systems with adaptive control mechanisms. However, existing methods^{4–9} for this problem do not have adequate robustness, flexibility or scalability.

Classical approaches predominantly rely on mapping-based methods for navigation^{5,6}. These methods typically divide the navigation task into different stages, such as perception, planning and control. Such methods have achieved impressive results, from exploring Mars¹⁰ to swarm navigation in dense forests⁷. However, the cascade structure introduces latency and errors accumulate, which limits their scalability to high-speed flight^{8,9} and dynamic environments. In multi-agent settings, explicitly constructing voxel grids to represent the environment and other agents adds further delay, making these approaches less suitable for agile swarm operations.

¹Shanghai Jiao Tong University, Shanghai, China. ²University of Zurich, Zurich, Switzerland. ³These authors contributed equally: Yuang Zhang, Yu Hu, Yunlong Song. ✉e-mail: dpzou@sjtu.edu.cn; wylin@sjtu.edu.cn

By contrast, recent studies have proposed learning end-to-end policies directly from data. For instance, reinforcement learning has achieved impressive results in solving complex robot control problems^{11–14}. Imitation learning has been widely used in vision-based applications due to its straightforward implementation and high sample efficiency. For instance, a study by ref. 15 proposed learning a sensorimotor policy that enables an autonomous quadrotor to fly acrobatic manoeuvres by leveraging demonstrations from an optimal controller.

Despite these advances, most existing methods focus on single-agent control and navigation, leaving multi-robot systems relatively unexplored. Demonstrations that have proven successful for individual drones are not readily applicable to swarms, primarily because coordinating several agents introduces substantial complexity. On the one hand, reinforcement-learning-based approaches require large amounts of training data, which grows exponentially with the number of robots and becomes exceedingly costly and time-consuming. On the other hand, imitation-learning-based methods rely on expert demonstrations, yet designing expert policies for swarms is far more challenging than for single agents. As a result, achieving agile swarm navigation remains an open research problem.

This work explores a different robot learning paradigm for swarm vision-based agile flight by training neural network policies using analytical policy gradients enabled through a differentiable-physics simulation. Specifically, we leverage prior knowledge about the physical system and its inherent differentiability to optimize a deep neural network policy end-to-end. Differentiable dynamics directly enables backpropagation through a physical simulator. This stands in contrast to model-free reinforcement learning and imitation learning; both methods treat the system dynamics as a black box and estimate the policy gradient using sampled data. We discovered that using physics priors for policy optimization enhances the training efficiency, task scalability and generalizability of the learned flight system.

Contributions

First, we introduce a learning-based method that leverages a differentiable simulation to effectively train a vision-based flight policy for a team of quadrotors to fly through complex unknown environments at high speed. This represents an advance in applying differentiable simulation to real-world vision-based drone navigation. Our method outperformed state-of-the-art vision-based flight systems in several simulation environments in terms of both success rate and agility. Compared to existing learning-based techniques^{4,8,16,17}, our solution demonstrated better training efficiency and reduced wall-clock time. We attribute these improvements to the use of analytical first-order gradients, which exhibit lower variance than zeroth-order gradient methods¹⁸.

Second, we demonstrate end-to-end, agile, vision-based, swarm navigation without explicit mapping and planning in dynamic and confined environments. After training in a multi-agent setting, the policy (deployed for decentralized control) can be used by several drones to navigate through narrow gates without explicit communication. The agents exhibit self-organized behaviours, such as waiting, following, resolving conflicts and mutual collision avoidance, which emerged without explicit collaboration design. These behaviours could inspire further research in this area.

Third, our system achieves exceptional computational efficiency. It runs on a budget-friendly US\$21 ARM-based computer with a depth camera. This is a crucial advantage over other state-of-the-art systems^{7,8,15,19} that rely on expensive and power-hungry graphics processing units (GPUs), which limits their use in resource-constrained environments.

Figure 1 provides an overview of our real-world experiments and demonstrates reliable high-speed flight in unstructured, dynamic environments. The policy, trained in simple simulation environments, generalizes well to real-world scenarios, achieving speeds of up to

20 m s⁻¹. This agility is attributed to our physics-informed learning approach and the flexibility of our end-to-end flight system design.

Results

Supplementary Video 1 summarizes the results and our method. In this section, we highlight the robustness, flexibility and generalizability of our method. We describe high-speed flight in complex and dynamic environments, agile communication-free swarm navigation and odometry-free flight and compare the method to benchmarks.

High-speed flight through complex and dynamic environments

We demonstrate the robust navigation performance of our end-to-end policy in complex and dynamic environments. To push the limit of vision-based agile flight in the real world, we built a lightweight drone. It had onboard a tiny low-cost computer for all computational tasks and an Intel RealSense depth camera (Fig. 2a). Our system weighed only 0.365 kg and had a thrust-to-weight ratio of 3.8.

Figure 2 summarizes our real-world results. We tested our system in various real-world environments, including dense forests, city parks and indoor environments with both static and dynamic obstacles. We designed the navigation tasks by specifying an initial position in an unknown environment. A trial was successful if the drone navigated through the obstacles without collisions. None of the testing environments had been seen by the system during training. Outdoor speeds were measured using GPS, whereas indoor speeds were from a motion capture system. We report the peak speeds during successful flights and success rates for different desired speeds in Fig. 2j,k. The results demonstrate the high agility of the system, which was achieved while ensuring safety. The supplementary videos are recordings of the flights.

Our policy generalized well to diverse real-world environments. Figure 2b,e shows a flight through a dense forest with low bushes. High-speed traversal was very difficult due to the low visibility and the complex distribution of fine branches. Figure 2c,d demonstrates flight at 7 m s⁻¹ through dense obstacles, and Fig. 2f shows flight at 20 m s⁻¹ in a forest.

Environments often change in real-world scenarios. This poses major challenges. For mapping-based methods, a system delay arises from their cascaded structure and the computational burden of map construction. For learning-based methods, the issue lies in their ability to generalize to unseen environments. Figure 2g–i illustrates the agility of our system in navigating through dynamic obstacles. Figure 2g shows navigation through a closing gate, Fig. 2h demonstrates safe flight near a moving obstacle and Fig. 2i captures swift responses to swinging obstacles. In the environments in Fig. 2d,f–h, the agent achieved 100% success across ten trials at 7 m s⁻¹. Although our network received one depth image per time step (along with the vehicle state), we employed a gated recurrent unit (GRU) layer. This recurrent component enabled the model to retain and update a hidden state across consecutive time steps. As a result, the network can capture motion cues from successive frames without requiring explicit multi-frame image inputs at a single forward pass. This enabled the network to perceive environmental changes over time and react to dynamic obstacles.

Agile communication-free swarm navigation

Swarm robotics is a multi-robot system that exhibits collective behaviour through local interactions and decentralized control, just like natural swarms of ants or bees. This approach is inherently robust, as the system continues to function even if individual agents fail or disconnect²⁰. The ability to deploy on low-cost hardware makes the swarm economically scalable²¹. We next show that our method can be directly used for swarm vision-based flight.

Figures 1 and 3 show how our approach enables several aerial robots to navigate cluttered environments populated by both static and moving obstacles, all without inter-robot communication. During

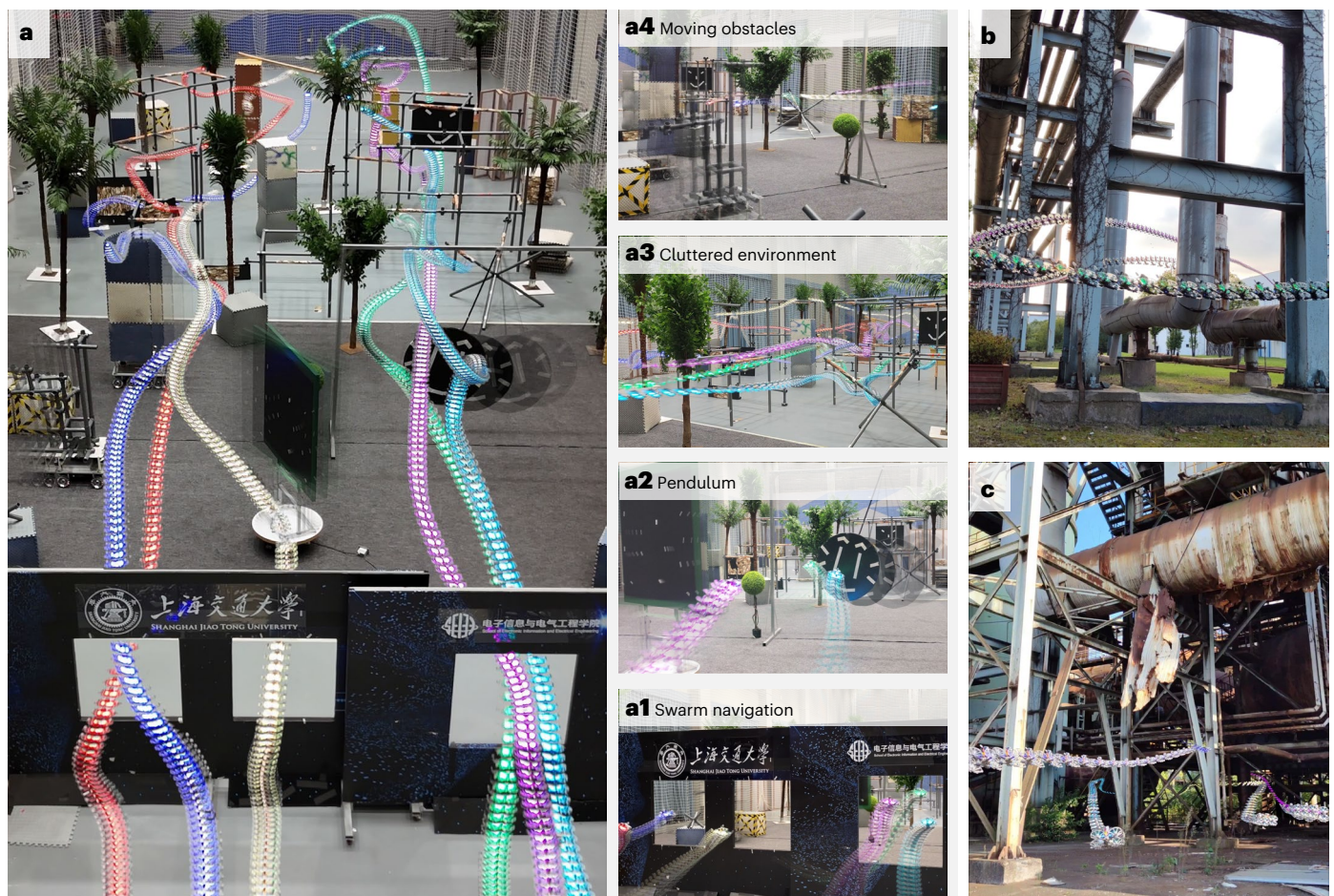


Fig. 1 | Vision-based agile swarm navigation through cluttered environments using an end-to-end neural network controller trained with differentiable physics. a, The indoor environment features both static and dynamic obstacles. **b**, Industrial metal framework. **c**, Abandoned factory. Our system operates on a

budget-friendly US\$21 ARM-based computer coupled with a depth camera for onboard sensing. All results were achieved using onboard vision sensors without communication between robots.

multi-agent training, each robot was modelled as a moving sphere in the perspectives of others. Once trained, the policy runs independently on each agent using identical observation modalities: depth, orientation and velocity.

To further demonstrate our approach, we designed a challenging task. A team of drones had to swap positions by navigating through a narrow gate in opposite directions. Figure 3a summarizes the result. We conducted the real-world experiments using six drones, which were divided into two groups of three agents each. Each group was positioned on opposite sides of the testing arena. The objective was for all six agents to navigate through a narrow gate to reach their destinations. Because of its small size, it was impossible for all six drones to pass through the gate simultaneously. Hence, the agents were required to coordinate with each other. To ensure that each agent could find its target position correctly, a motion capture system provided each agent with (1) the target direction and (2) the velocity estimate as the input to the neural network. Each agent received only its own velocity and target direction within its own body frame. No information about the other agents was shared. Therefore, the agents had to execute individual actions without communicating with each other. The result shows that all six drones successfully passed through the gate and reached their destination.

Interestingly, the trained vision-based agents demonstrated emerging self-organized behaviours—such as waiting in air traffic, following an agent and retreating to give way to incoming agents—all without the need for communication, global planning or coordination loss in training.

Furthermore, we quantitatively compare our approach with the state-of-the-art swarm planning method, Ego v.2 (ref. 7), in terms of success rates and task completion time. A trial was considered successful if all drones reached their goals within 1.2 m without collisions. Figure 3d,e shows that our communication-free approach achieved success rates comparable to those of Ego v.2 (ref. 7). In position-swapping, our method took longer (median of 4.7 s versus 4.0 s for Ego v.2) due to cautious lateral movements. However, in parallel flight, our method completed tasks faster (median of 6.5 s versus 9.0 s for Ego v.2), demonstrating its enhanced agility. Additionally, we compared our approach to a proximal policy optimization (PPO) baseline trained with the same reward (negative loss) and environment. Although PPO had faster task completion times, it achieved lower success rates of 30–70% in the parallel flight and of 10–30% in the position-swapping scenarios. Furthermore, we did not observe PPO exhibit multi-agent behaviours (for example, waiting, retreating and following), as demonstrated by our approach.

Notably, existing methods rely heavily on network communication and global planning for coordination and mutual avoidance. However, communication can be unreliable, and centralized planning requires accurate relative positioning, which is difficult to acquire. Consequentially, Ego v.2 adopts ad hoc ultra-wideband positioning and drone detection in real-world experiments.

Much like humans or other animals, our agents achieve self-organization using only local sensing. This enables their deployment in environments without global positioning or communication

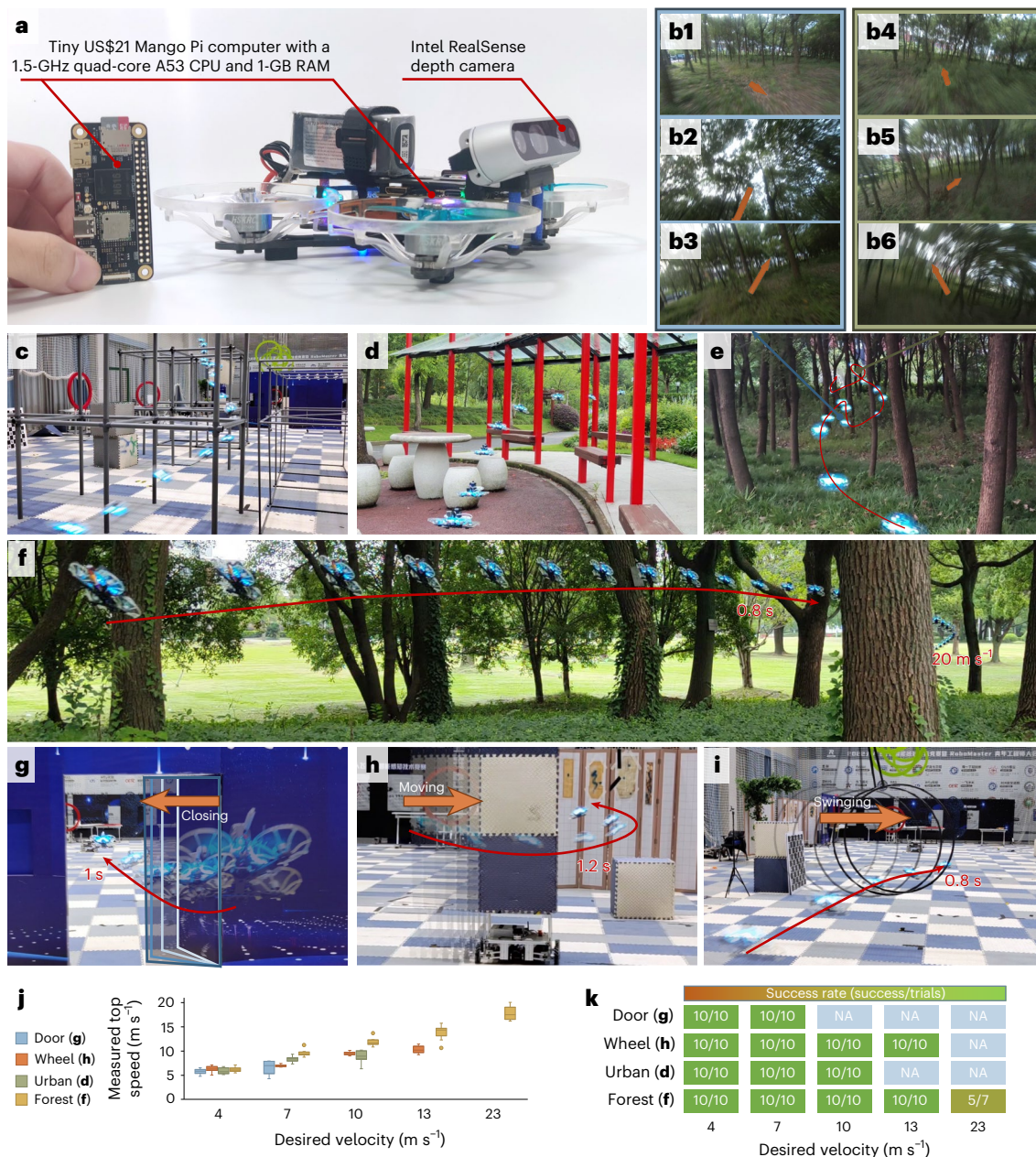


Fig. 2 | High-speed flight through complex and dynamic environments using a tiny US\$21 low-cost computer. **a**, Experimental platform. **b–i**, We demonstrate the robustness of navigating in dense forests, urban environments and dynamic obstacles. **b**, A dense forest (first-person views). **c**, Scaffold. **d**, Urban park. **e**, Dense forest. **f**, High-speed flight in a forest. **g**, Closing door. **h**, Translational obstacle. **i**, Swinging ring. **j**, Box plot of speeds achieved in successful trials. The centre line represents the median, the box limits correspond to the upper and

lower quartiles, the whiskers extend to 1.5 times the interquartile range, and the points indicate outliers. Each sample represents an independent successful flight trial. The sample sizes are the corresponding numbers of successful trials shown in **k**. **k**, Number of successful and total trials. All these experiments involved end-to-end navigation without reliance on visual or inertial odometry or GPS localization for state estimation. NA, not applicable.

infrastructure, such as in search and rescue missions or remote locations. Our approach provides a robust solution for swarm navigation and obstacle avoidance in these challenging conditions.

Odometry-free flight

To further evaluate the flexibility and robustness of our end-to-end flight system, we studied the role of state estimation in obstacle avoidance. Existing approaches rely heavily on the quality of visual–inertial odometry, which is sensitive during high-speed flight^{22–24}. High-speed motion can make feature-tracking difficult²², resulting in the failure of the visual localization system. Additionally, visual localization and mapping demand considerable computational resources. For instance,

the CPU usage of Ego v.2 for localization exceeds the combined usage of all other processes, even with GPU acceleration⁷.

Our approach allows agents to be trained without position and velocity estimates, resulting in odometry-free agents that rely solely on attitude estimation and depth maps for navigation. Our method uses a recurrent neural network (RNN) that maintains a hidden state across consecutive time steps (Extended Data Fig. 1), thus enabling the system to operate without explicit odometry for state estimation when deployed on a real drone. In this way, we further exploit the potential of neural networks to achieve end-to-end localization, planning and control.

We assessed the performance of our odometry-free agent by comparing its success rate and flight speed against our baseline agent, which

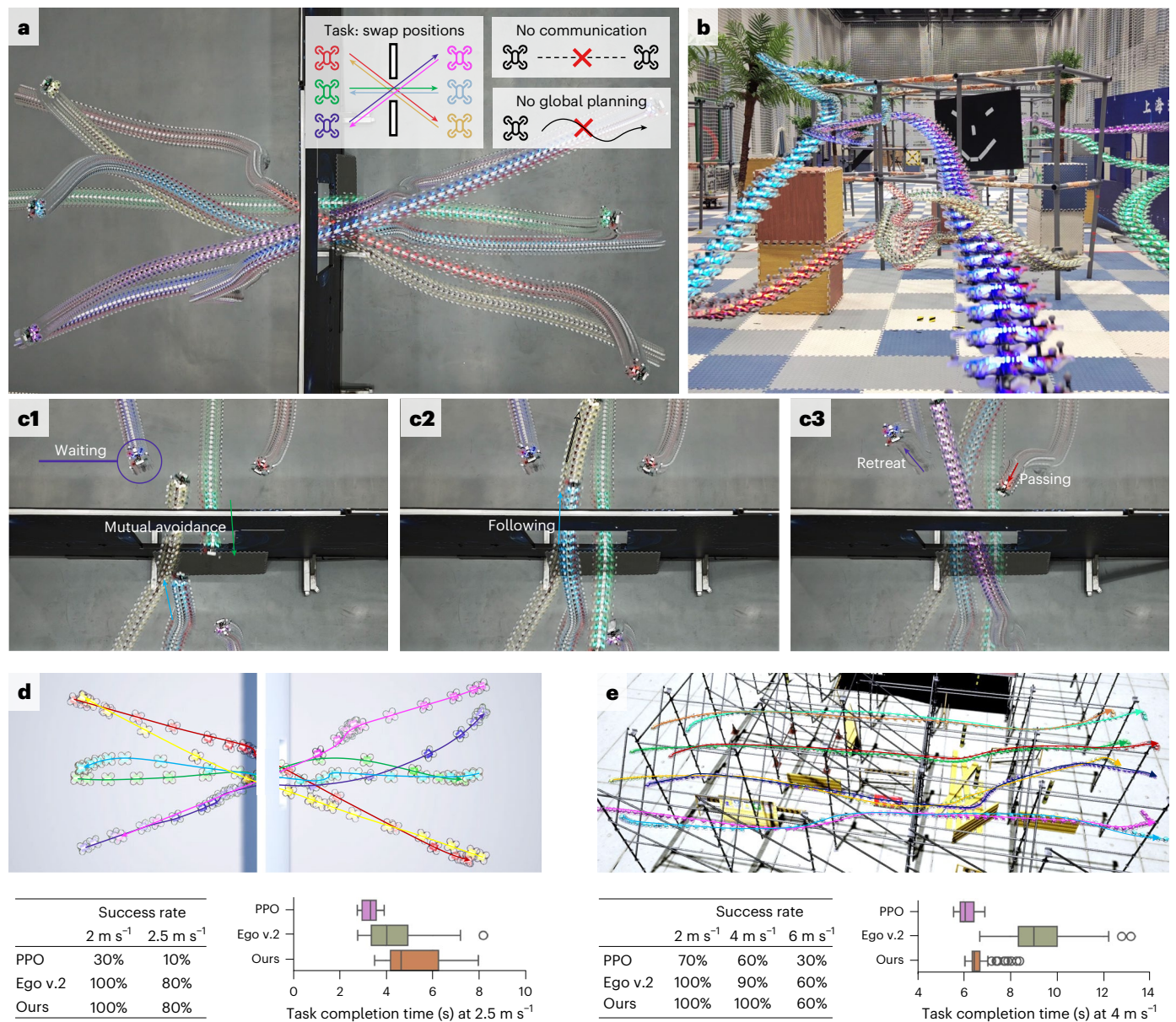


Fig. 3 | Communication-free swarm navigation. **a, b**, Real-world evaluations with six drones navigating through a narrow gate in opposite directions (**a**) and various obstacles in the same direction (**b**). **c**, Enlarged views showing the emergence of collective intelligence. Video demonstrations are available. **d, e**, Simulated tests: six drones swapping positions through a gate (**d**) and eight drones flying in parallel (**e**). The experiments were replicated 10 times with the initial position perturbed within a 0.1-m cube. Box plots show the task completion times. Each sample represents an independent successful agent

among all replicates and all agents. Sample sizes for **d** were $n = 38$ for PPO, $n = 58$ for Ego v.2 and $n = 56$ for our system. For **e**, sample sizes were $n = 74$ for PPO, $n = 78$ for Ego v.2 and $n = 80$ for our system. In the box plots, the centre line represents the median, the box limits correspond to the upper and lower quartiles, the whiskers extend to 1.5 times the interquartile range, and points beyond the whiskers indicate outliers. Our success rates match those of Ego v2 (ref. 7, with communication and global positioning) while preserving our high-speed flight advantage. Conversely, PPO¹⁸ had lower success rates.

used external velocity estimation inputs (odometry-based) in real-world scenarios. We evaluated the odometry-based agents with two velocity estimation sources: a Vicon motion capture system and the visual-inertial odometry system Vins-Fusion²⁵. The testing environment, as shown in Fig. 4a, includes an open area at the front designed to enable the drone to achieve high speeds, along with obstacles, such as cubes and trees, at the rear. We evaluated the success rate (Fig. 4b) and speed (Fig. 4c) of the drones in reaching the area behind the obstacles. We tested at 3 desired speeds, 4, 7 and 10 m s⁻¹, repeating each test 10 times. The agent using visual-inertial odometry had a lower success rate of 90% at 7 m s⁻¹ and 80% at 10 m s⁻¹, consistent with the limitations of visual-inertial odometry. Our odometry-free agent matched the performance of the Vicon-based agent and consistently reached the targets

without crashing. This indicates that the odometry-free agent may have reliable implicit state estimation. To test this, we retrained the RNN to predict the vehicle velocity. Figure 4d compares the network-predicted velocity with the ground truth velocity from the motion capture system for a real-world flight sample. During the sample flight, which had a maximum speed of 7.74 m s⁻¹, the average velocity prediction error $\|v_{\text{pred}} - v_{\text{real}}\|$ was 0.78 m s⁻¹. The predicted velocity aligned with the ground truth closely, which shows the capability of the neural network in state estimation. Furthermore, to validate the effectiveness of our method in visual perception, we used gradient-weighted class activation mapping (Grad-CAM)²⁶ to visualize which parts of the depth image contribute to the action. Specifically, we employed the squared magnitude of lateral

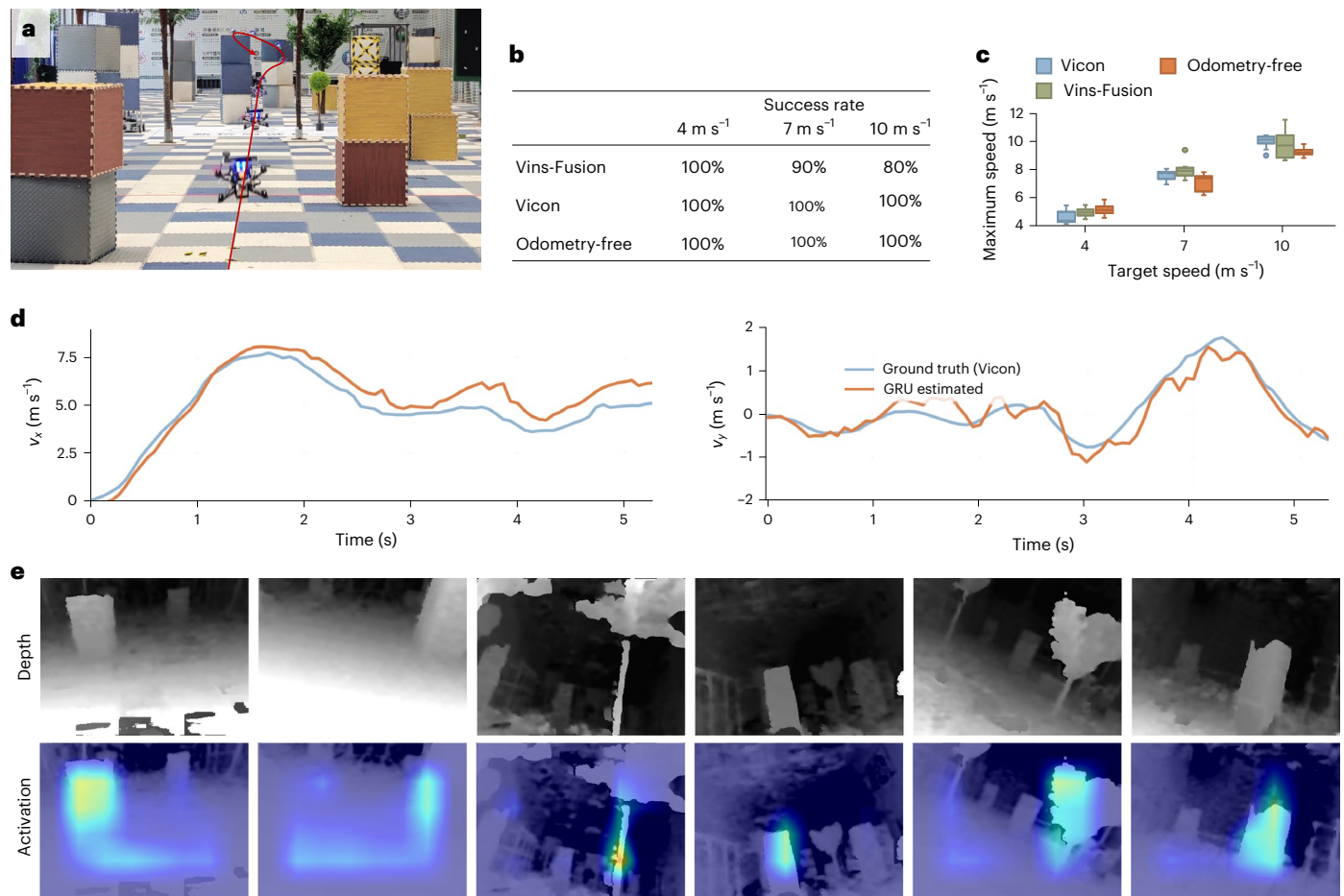


Fig. 4 | End-to-end vision-based flight without an explicit odometry module. **a**, Testing environment. **b**, Success rates at different target speeds. We conducted ten flight trials for each target speed. Our odometry-free system matched the performance of the Vicon-based agent. **c**, Box plots of the maximum speeds. Each sample represent an independent successful tail. For Vins-Fusion, sample sizes were $n = 10$ at 4 m s⁻¹, $n = 9$ at 7 m s⁻¹ and $n = 8$ at 10 m s⁻¹. The sample sizes for

Vicon and odometry-free were ten. The centre line represents the median, the box limits correspond to the upper and lower quartiles, the whiskers extend to 1.5 times the interquartile range, and points beyond the whiskers indicate outliers. **d**, Comparisons of the ego velocity predicted by the neural network and the ground truth during a sample flight. **e**, Activation maps during the sample flight.

acceleration (a_y^2) as our loss function for Grad-CAM visualization, where a_y denotes the lateral acceleration in the body frame. We visualized the activations from the second leaky ReLU layer in the convolutional neural network. Figure 4e shows the activation maps during a sample flight. The activated regions correspond to obstacles that pose direct collision threats. The full sequence is available in Supplementary Video 8.

Baseline comparisons

This section provides a thorough evaluation, including baseline comparisons with several state-of-the-art methods known for impressive vision-based real-world flight results. We also measured our approach against both reinforcement learning and imitation learning in terms of training efficiency and wall-clock time. Finally, we used ablation studies to validate the key design decisions.

Comparison to state-of-the-art methods. We tested the obstacle avoidance performance in three sets of simulated environments: a forest (Fig. 5a), urban areas (Fig. 5b) and in factories (Fig. 5c).

To ensure a fair comparison with Agile⁸, we first adopted experimental settings aligned with their approach using the forest scene in the Flightmare simulator²⁷, as shown in Fig. 5a. The results for the mapping-based baseline FastPlanner²⁸, the motion primitive-based method Reactive²⁹ and the Agile approach were all sourced from the Agile paper⁸. To supplement this with more test environments, we conducted

other tests in urban and factory environments using the AirSim simulator³⁰, as shown in Fig. 5b,c. The comparison methods include the mapping-based method Ego v.2 (ref. 7) and the learning-based method Agile⁸, using their publicly available code and pretrained weights.

We tested all methods at different target speeds: 4, 7, 10, 13 and 22 m s⁻¹. Each experiment was repeated ten times with different initial positions, and we computed the average flight speed, peak flight speed and success rate. A task was considered to be successful if the drone reached its goal position within a 5-m radius without crashing. The average speed statistics are based on the successful trials.

The results are summarized in Fig. 5. The mapping-based method Ego v.2 (ref. 7) performed relatively well at very low speeds, for example, below 3 m s⁻¹; however, it experienced performance drops as the target speed was increased. This aligns with the real-world experiments demonstrated in their paper, in which the average flight speed was around 2 m s⁻¹. Our findings also confirm the results reported in ref. 8, where they attribute such failures to the increased processing and control latency inherent in cascaded structures. Additionally, we trained a reinforcement-learning baseline with the PPO algorithm¹⁸ using an identical reward and environment but without the differentiable simulation. Our approach achieved higher success rates than PPO across all speeds. Although the learning-based method Agile⁸ performed well in the forested environment, this specific environment was also used to train its navigation policy. When tested in an unseen environment, for

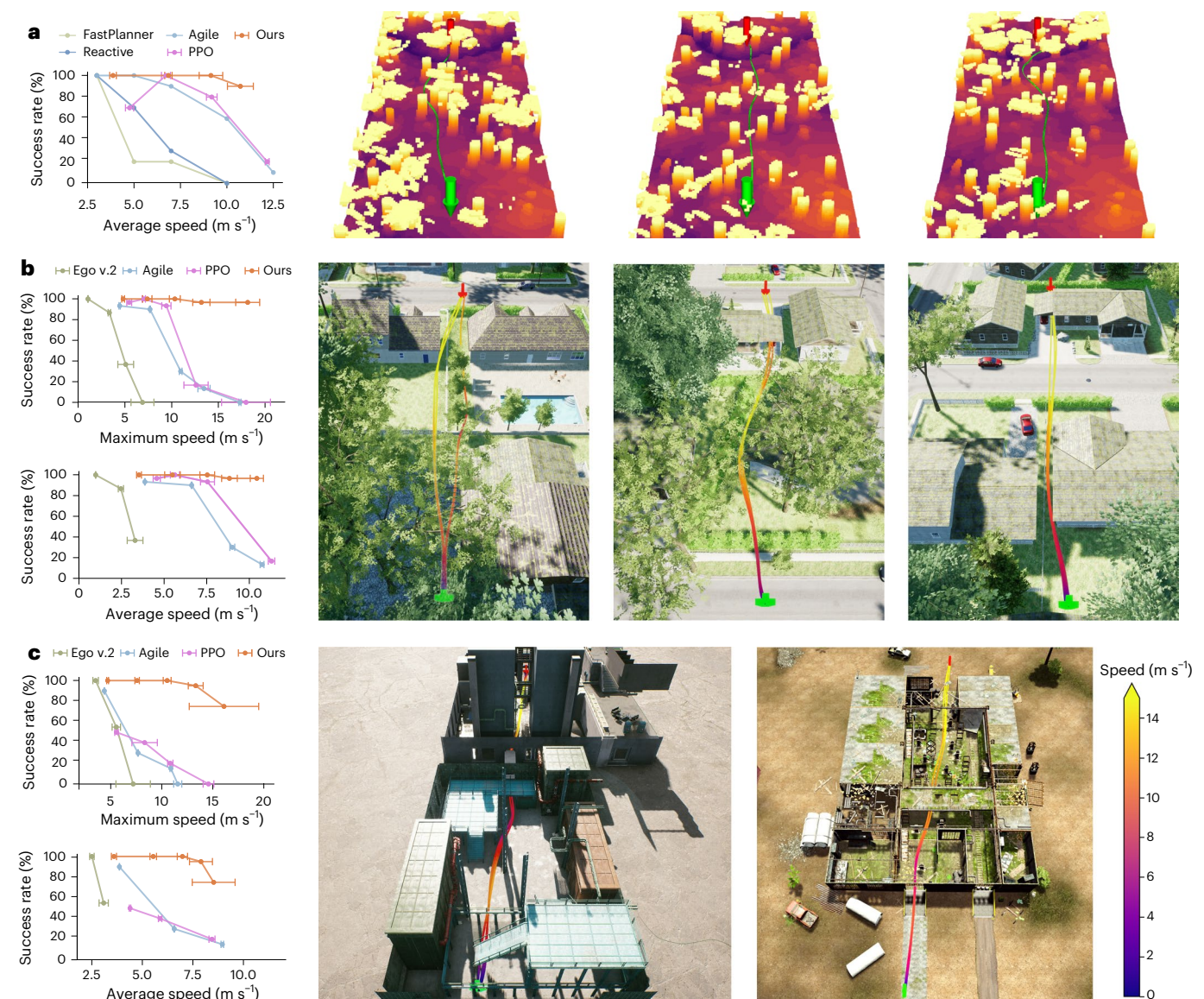


Fig. 5 | Comparison to state-of-the-art vision-based navigation. **a–c**, Test environments were a random forest (**a**), three urban environments (**b**) and two factory environments (**c**). Each experiment was repeated ten times per

environment and speed. Error bars represent the standard deviation of the achieved maximum speed or average speed given a fixed reference speed across all trials. Our method achieved the highest success rates across all environments.

example, in an urban setting⁸ (Fig. 5b,c), its success rates were lower. This indicates that expert-based approaches may face challenges when generalizing to unseen environments.

This shortcoming is primarily due to the difficulty in obtaining high-quality expert labels in obstacle-dense, high-speed conditions⁸. The sampling-based expert faces challenges in consistently identifying the optimal route, leading to erratic flight patterns in the trained agents. The incorporation of error corrections through the DAGger framework³¹, along with frequent trajectory replanning, exacerbates the unstable behaviour of the drone. Last, the expert-based approach tends to maintain a constant speed, as it lacks the flexibility to adapt its flight speed in response to varying obstacle densities.

Our approach achieves high success rates at high speeds (90% above 10 m s⁻¹; Fig. 5a) while eliminating the need for expert demonstrations. The end-to-end design enables stable control and automatic speed adaptation to obstacle density.

Training efficiency comparison to imitation learning. To assess whether differentiable physics converged faster than imitation learning

(particularly Agile⁸), we conducted training experiments using identical forest environments from the Agile framework. Because the loss functions differed between approaches, we evaluated performance using success rates measured at equivalent training steps and batch sizes, with testing conducted at 7 m s⁻¹ in the same environment. The results in Fig. 6a show the faster convergence of our method. By the 1,000th iteration, our approach had achieved a 100% success rate, whereas Agile⁸ and PPO¹⁸ were only starting to show success, with 20% and 30% success rates, respectively.

Comparison of data efficiency for reinforcement learning. To further examine the sample efficiency of differentiable physics, we evaluated our approach against the state-of-the-art reinforcement-learning algorithm PPO¹⁸ and Deep-ILC³² using the same training environment and reward. We compared PPO and our approach using both our recurrent policy based on GRU³³ and a non-recurrent multi-layer perceptron (MLP) policy (created by zeroing hidden states of the GRU policy). Figure 6b compares the learning curves. Our method required only 10% of the samples needed by the PPO algorithm to achieve its maximum

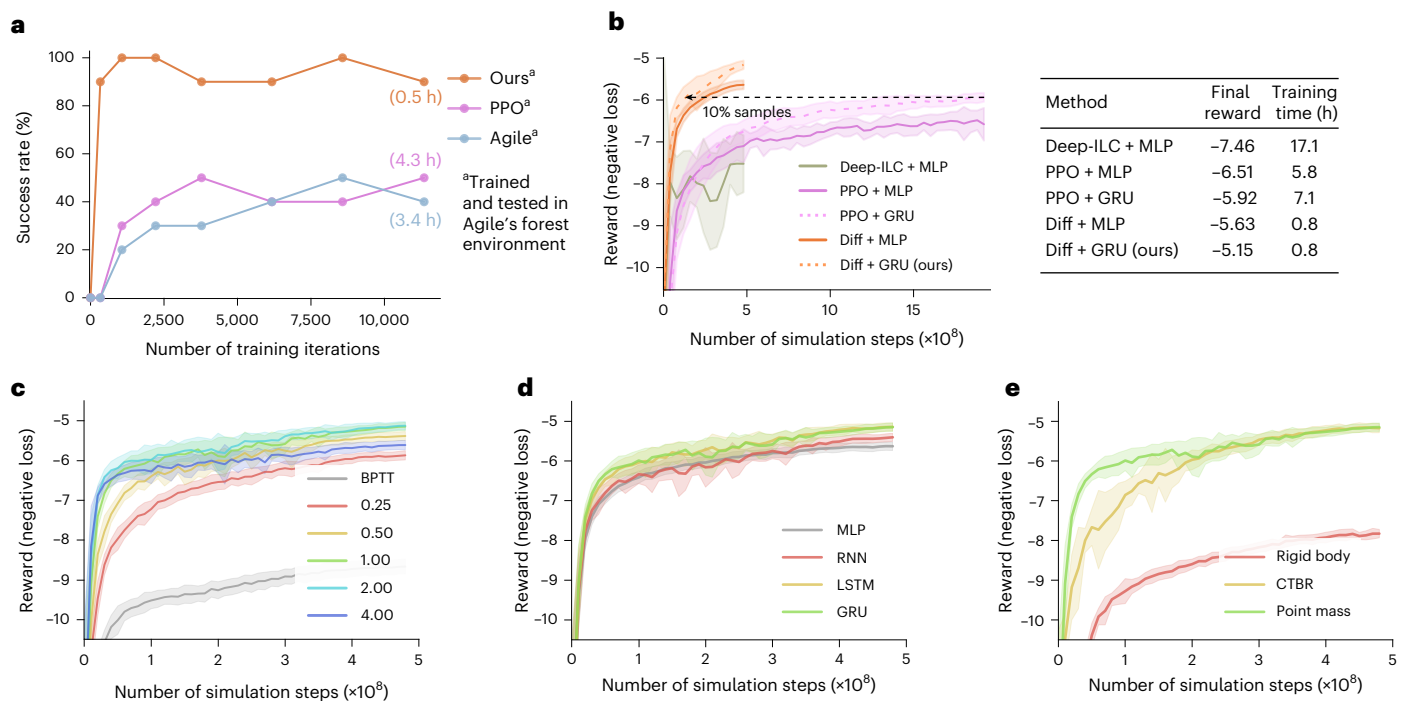


Fig. 6 | Ablation studies. **a, b**, Comparison of training efficiency between our method and the imitation-learning baseline Agile⁸ (**a**) and reinforcement-learning baselines PPO¹⁸ and Deep-ILC³² (**b**). In **a**, we report training times on a GTX 1080 GPU in parentheses. In **b**, we report training times on an RTX 4090 GPU.

c–e, Ablation studies of the temporal gradient decay (**c**), the neural network architecture (**d**) and the physical model (**e**). All models were trained with five random seeds. Mean reward curves are plotted with shaded areas showing the standard deviation. Diff, differentiable physics; LSTM, long short-term memory.

reward level, and ultimately, it achieved a higher value. We attribute this improved convergence to the lower variance of first-order gradients inherent in the smooth and well-behaved point-mass model³⁴. Furthermore, Fig. 5a–c shows that our approach achieved higher success rates across all velocities tested in the simulation.

Notably, Deep-ILC uses soft actor-critic³⁵ with simulator gradients to enhance critic training. However, it relies on the critic model to optimize policy, which might cause inaccurate policy updates due to critic model errors. By contrast, our approach directly uses simulator gradients for policy optimization. Although Deep-ILC initially matched the sample efficiency of our system by quickly reaching a reward of -8, it later oscillated between -9 and -7 and converged at -7.46. In comparison, differentiable physics achieved stable convergence to a superior reward of -5.63.

Ablation study. Although leveraging differentiable physics is the key aspect of our approach, directly using the gradient through the physics did not work properly, primarily due to gradient explosion, as pointed out in ref. 36. We addressed this by applying exponential gradient decay $e^{-\alpha \Delta t}$ during backpropagation, as visualized in Extended Data Fig. 2 (see also ‘Temporal gradient decay’). As shown in Fig. 6c, the gradient decay mechanism ($\alpha \neq 0$) accelerated convergence compared to the vanilla backpropagation through time (BPTT). The training performance was relatively insensitive to α , with values from 0.25 to 4 all substantially outperforming BPTT.

Our approach uses GRU³³ to process temporal input sequences. With gradient decay applied, GRU achieved higher rewards than a standard MLP and simple RNN³⁷, and performed like long short-term memory³⁸ (Fig. 6d). We selected GRU for its simplicity.

To evaluate the impact of the point-mass model on performance, we tested two higher-fidelity alternatives: the collective thrust body rate (CTBR) model, where the policy outputs the desired collective thrust and body rates, and the torque-thrust rigid-body model, where the policy generates the desired torque and collective thrust. All

models were trained using differentiable physics with the same loss function. Figure 6e shows that the point-mass model achieved the best asymptotic performance while requiring the fewest simulation samples, whereas CTBR converged more slowly but reached comparable results. This highlights the importance of using differentiable physics for training, as even a basic physics prior can improve performance and training efficiency.

Discussion

This work has demonstrated the effectiveness of differentiable physics in agile vision-based navigation. Unlike reinforcement learning and imitation learning, which often treat the physics model in the robot as a black box, our approach leverages a differentiable-physics simulation to train a neural network flight policy. This enables end-to-end optimization by backpropagating loss gradients directly through the physics model used by the robot. As a result, our methodology eliminates the need for task-specific expert labels. Compared to reinforcement-learning methods, which rely on zeroth-order optimization, differentiable physics uses gradient information for first-order optimization, leading to improved sample efficiency.

Our method achieved exceptional real-world, vision-based flight with zero-shot transfer and robust scalability for multi-agent scenarios. Experiments with swarms of quadrotors demonstrated agile navigation in dynamically changing environments and confined spaces, even in the absence of explicit communication. The policy exhibited coordinated behaviours such as waiting and following, and its success in partially observable settings reduces system complexity while enabling deployment in unpredictable environments. Furthermore, we validated our approach under extremely constrained sensory and computational resources.

Limitations and future research

There are several options for future research. First, the limited field of view remains a challenge. Although our predefined heading for

odometry-free flight demonstrates strong performance, it may limit long-term navigation where yawing may be required. During rapid acceleration or deceleration, the camera can shift towards the ground or sky, compromising obstacle perception. In swarm applications, there could be collisions with agents outside the field of view (Supplementary Video 9), which could be addressed with a wide field-of-view camera or several cameras. Second, although our point-mass model enables efficient training and simulation-to-real transfer, using a full rigid-body model could enable more advanced manoeuvres, such as aerobatic flight. Finally, the air drag parameters require recalibration across platforms. Future work could focus on online air drag calibration and dynamic adjustment³⁹.

Method

Problem formulation

Without loss of generality, we formulated the robot navigation task in an optimization framework. The robot is modelled as a discrete-time dynamical system, characterized by continuous state and control input spaces, denoted as $\mathcal{X}$ and $\mathcal{U}$, respectively. At each time step k , the system state is $\mathbf{x}_k \in \mathcal{X}$, and the corresponding control input is $\mathbf{u}_k \in \mathcal{U}$. An observation $\mathbf{o}_k \in \mathcal{O}$ is generated at each time step based on the current state $\mathbf{x}_k$ through a sensor model $h: \mathcal{X} \rightarrow \mathcal{O}$, such that $\mathbf{o}_k = h(\mathbf{x}_k)$. The system dynamics are governed by the function $f: \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$, which describes the time-discretized evolution of the system as $\mathbf{x}_{k+1} = \mathbf{x}_k + f(\mathbf{x}_k, \mathbf{u}_k)$. The discrete-time instants are $t_k = \Delta t \times k$, where k ranges from 0 to N , thereby establishing a finite time horizon for the control problem. Crucially, at each time step k , the robot receives a cost signal $l_k = l(\mathbf{x}_k, \mathbf{u}_k)$, which is a differentiable function of the current state $\mathbf{x}_k$ and the control input $\mathbf{u}_k$.

We model the policy as a function approximator represented by a neural network $\mathbf{u}_k = \pi_{\theta}(\mathbf{o}_k)$. The neural network takes the observation $\mathbf{o}_k$ as input and outputs the control input $\mathbf{u}_k$. The objective of our optimization problem is to find the optimal policy parameters θ^* by minimizing the total loss using gradient descent:

$$\min_{\theta} \mathcal{L}_{\theta} = \sum_{k=0}^{N-1} l(\mathbf{x}_k, \mathbf{u}_k) = \sum_{k=0}^{N-1} l(\mathbf{x}_k, \pi_{\theta}(\mathbf{o}_k)), \quad (1)$$

$$\theta \leftarrow \theta - \gamma \nabla_{\theta} \mathcal{L}_{\theta}, \quad (2)$$

where γ is the learning rate.

Policy gradient from a differentiable simulation

In a differentiable simulation for policy-learning, the backward pass is crucial for computing the analytic gradient of the objective function with respect to the policy parameters. Following ref. 36, the policy gradient can be computed directly:

$$\frac{\partial \mathcal{L}_{\theta}}{\partial \theta} = \frac{1}{N} \sum_{k=0}^{N-1} \left(\sum_{i=0}^k \frac{\partial l_k}{\partial \mathbf{x}_i} \prod_{j=i+1}^k \left(\frac{\partial \mathbf{x}_j}{\partial \mathbf{x}_{j-1}} \right) \frac{\partial \mathbf{x}_i}{\partial \theta} + \frac{\partial l_k}{\partial \mathbf{u}_k} \frac{\partial \mathbf{u}_k}{\partial \theta} \right),$$

where the matrix of partial derivatives $\partial \mathbf{x}_j / \partial \mathbf{x}_{j-1}$ is the Jacobian of the dynamical system f . Therefore, we can compute the policy gradient directly by backpropagating through the differentiable-physics model and a physics-driven objective function l_k that is differentiable with respect to the physics model. This enables more efficient and potentially more accurate optimization compared to traditional reinforcement-learning methods.

Differentiable-physics simulation

The fundamental difference between a differentiable-physics simulation and a standard simulation, for example MuJoCo⁴⁰ or Flightmare²⁷, lies in its ability to compute gradients with respect to model parameters

or inputs. Such a simulation can propagate the gradient from the output state $\mathbf{x}_k$ directly to the control input $\mathbf{u}_k$, and hence, to the policy parameters θ .

We produced a differentiable-physics simulation for quadrotors. Typically, a quadrotor is modelled as a rigid body that is actuated by four motors²⁷. Instead of seeking to accurately model the real-world quadrotor dynamics, we opted to use a simple point-mass model to approximate the quadrotor dynamics. Although common belief suggests that high-fidelity models are necessary for a successful simulation-to-reality transfer, our approach demonstrates that this simple model can achieve this goal. Additionally, this approximation enables faster gradient computation and a better optimization landscape for training, thus improving efficiency. Thus, we have based our differentiable-physics simulation on a simple point-mass model, which is represented by the numerical integration of acceleration and velocity:

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \frac{\mathbf{a}_k + \mathbf{a}_{k+1}}{2} \Delta t, \quad \mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k + \frac{1}{2} \mathbf{a}_k \Delta t^2.$$

where $\mathbf{a}_k$ is acceleration, $\mathbf{v}_k$ is velocity and $\mathbf{p}_k$ is position. We used an acceleration-based controller that converts the acceleration outputs of the policy into a desired attitude and collective thrust, which are then tracked in real time by a high-frequency inner-loop flight controller⁴¹. To model the flight controller response, we implemented a fixed control latency and an exponential moving average on the sequence of desired thrust acceleration values. This process, illustrated in Extended Data Fig. 1, can be represented as the convolution between the desired thrust vector $\mathbf{u}_k$ with the simplified flight controller impulsive response:

$$\eta(t) = \begin{cases} 0, & t < \tau, \\ \lambda e^{-\lambda(t-\tau)}, & t \geq \tau, \end{cases}$$

where τ is a fixed delay and λ is the exponential smoothing factor. The filtered thrust $\hat{\mathbf{u}}_k$ is considered as the actual thrust applied on the point-mass model.

We also added a quadratic air drag and took the sum of thrust and air drag to calculate the acceleration ($\mathbf{a}_k$) for the point-mass physics simulation. The parameters τ and λ as well as the air drag coefficients are tuned to match the real platform through calibration (Supplementary Note 1). Because a quadrotor can generate only a collective thrust in the upward direction for its body, the magnitude (length) of the thrusts defines the overall vehicle acceleration, whereas the direction of the thrust vector defines the inclination. Additionally, we aligned the yaw angle of the quadrotor with the target direction, thus establishing a unique orientation. After that, the orientation and position of the quadrotor (from the point-mass model) served as the extrinsic parameters for rendering the depth sensor data during the training phase.

Training environment and physics-driven objective function

Our training environment, including depth-rendering and the physics simulation, was implemented with CUDA as a PyTorch extension³⁷. Building upon the set-up of the first-order end-to-end policy training system in this work, our subsequent research⁴² used a similar methodology to investigate a distinct question related to optical flow-based obstacle avoidance. As illustrated in Extended Data Fig. 1, the training environment features four kinds of simple geometric obstacles: planes, cuboids, spheres and cylinders. Although existing methods for training vision-based policies typically rely on high-fidelity simulators to generate realistic scenes, we discovered that our training approach, even when using such a simple simulation environment, can produce robust vision-based flight policies that generalize well to diverse real-world scenarios. We randomly generated the dimensions and positions of the obstacles for each roll-out. For one training batch, there were 64

environments running in parallel. At each training iteration, we collected 64 trajectories by simulating the environments for 150 time steps, with a time step of 1/15 s.

The physics-driven objective function (or loss) consists of three parts: matching the desired velocities, obstacle avoidance and control smoothness. The velocity matching error is measured using the smooth L1 loss⁴³, a robust loss less sensitive to noisy fitting errors, between the velocity control error and zero,

$$\mathcal{L}_v = \frac{1}{T} \sum_{k=1}^T \text{Smooth L1}(\|\mathbf{v}_k^{\text{set}} - \bar{\mathbf{v}}_k\|_2, 0)$$

where T is the total number of time steps and $\bar{\mathbf{v}}_k$ is the average velocity calculated as a moving average across a 2-s time window. The averaging enables the agent to focus on obstacle avoidance locally while tracking the desired velocity $\mathbf{v}_k^{\text{set}}$ in a longer horizon. The direction of the desired velocity, denoted as $\mathbf{v}_k^{\text{set}}$, is determined by the difference between the target position and the current position. Its magnitude is limited to a predefined maximum speed, and its gradient is not propagated during training.

The obstacle avoidance loss is defined by the distance between the quadrotor centre and the nearest point in the environment. We denote the closest obstacle distance as d_k , the radius of the drone as r_q and the speed of approach to the closest obstacle point as v_k^c . The obstacle avoidance loss is the approach speed multiplied by the sum of a truncated quadratic function and a softplus function⁴⁴:

$$\mathcal{L}_c = \frac{1}{T} \sum_{k=1}^T \left[v_k^c \max(1 - (d_k - r_q), 0)^2 + \beta_1 \ln(1 + e^{\beta_2(r_q - d_k)}) \right],$$

where β_1 and β_2 are control parameters for the softplus function. The softplus function represents a penalty term that increases smoothly as the quadrotor collides with obstacles, so that it effectively acts as a barrier function. This function helps to enforce a constraint on the proximity of the quadrotor to obstacles in a continuous and differentiable manner. The parameter β_1 controls the weight of this penalty, and β_2 controls its steepness. We chose $\beta_1 = 2.5$ and $\beta_2 = 32$ for our implementation. The approach speed v_k^c adjusts the penalty according to how fast the drone is approaching an obstacle. When the drone is stationary or moving away from the obstacle, this factor will nullify the penalty. Conversely, when the drone is moving towards the obstacle, it ensures that the obstacle avoidance loss is enough to slow down the agent.

To achieve dynamic feasibility and a smoother flight, we adopted a penalty on both the acceleration and jerk⁶:

$$\mathcal{L}_a = \frac{1}{T} \sum_{k=1}^T \|\mathbf{a}_k\|^2, \quad \mathcal{L}_j = \frac{1}{T-1} \sum_{k=1}^{T-1} \left\| \frac{\mathbf{a}_k - \mathbf{a}_{k+1}}{\Delta t} \right\|^2.$$

The final loss is the weighted sum of these four terms, that is, $L = \lambda_v \mathcal{L}_v + \lambda_c \mathcal{L}_c + \lambda_a \mathcal{L}_a + \lambda_j \mathcal{L}_j$, where $\lambda_v, \lambda_c, \lambda_a$ and λ_j are the weights. We chose $\lambda_v = 1, \lambda_c = 2, \lambda_a = 0.01$ and $\lambda_j = 0.001$ for our implementation.

Temporal gradient decay

Extended Data Fig. 2 presents the computation graph for the forward simulation and backpropagation of the gradients of the point-mass physics simulation. Although the physics simulation is differentiable, direct backpropagation leads to gradient explosion^{36,45} when the gradient is backpropagated through the entire simulation. This results in the issue where long-range gradients from future time steps can accumulate to large values, resulting in an unstable optimization. Intuitively, the agent can learn to avoid only relatively close obstacles, and the long-range gradients force the agent to avoid all future obstacles without considering the limitations of its perceptual range.

To overcome this problem, we incorporated an exponential decay mechanism into the position and velocity gradients during backpropagation. Although alternative strategies, such as reducing the number of simulation steps in the gradient computation, may also mitigate this issue, we focused on the decay mechanism due to its simplicity and effectiveness. Specifically, we multiply the gradient of $\mathbf{x}_{t+1}$ with respect to $\mathbf{x}_t$ by the exponential decay factor $e^{-\alpha \Delta t}$. The parameter α controls the decay rate (see Fig. 6c for the ablation study), and Δt is the duration of a single time step in the numerical simulation. This mechanism decays the gradient during backpropagation through time, producing two key effects: it mitigates gradient accumulation and ensures that the supervisory signal stems from the near future. By emphasizing immediate objectives and filtering out potential distractions from uncertain future obstacles, the adjusted gradient substantially boosted training performance. As a result, we optimized the policy using the following gradient, which has a temporal gradient decay:

$$\frac{\partial \mathcal{L}_\theta}{\partial \theta} = \frac{1}{N} \sum_{k=0}^{N-1} \left(\sum_{i=0}^k \frac{\partial l_k}{\partial \mathbf{x}_k} \prod_{j=i+1}^k \left(\frac{\partial \mathbf{x}_j}{\partial \mathbf{x}_{j-1}} e^{-\alpha \Delta t} \right) \frac{\partial \mathbf{x}_i}{\partial \theta} + \frac{\partial l_k}{\partial \mathbf{u}_k} \frac{\partial \mathbf{u}_k}{\partial \theta} \right).$$

Network architecture

We used a convolutional RNN⁴⁶ as the policy network. It combines a lightweight convolutional neural network to extract depth map features and a GRU³³ layer to capture temporal information. The inputs to our policy were depth images and state information. Depth images are crucial for providing general obstacle information (for example, distances to surfaces), which is essential for obstacle avoidance. States are necessary to ensure sufficient control authority to stabilize the drone and for it to follow a reference velocity. The depth map is inverted and max-pooled to 16×12 as network input (see Supplementary Fig. 4 for a visualization of pooled depth maps). Each frame passed through several convolutional neural network layers with filters (kernel size) 32(2), 64(3) and 128(3) and then flattened and linearly projected to a 192-dimensional feature. All the convolution layers had a stride of 1 and were followed by a leaky ReLU activation function. Moreover, the target velocity, attitude estimation and velocity estimation (optional) were linearly projected to a 192-dimensional feature and fused with the image feature through element-wise addition. The combined feature was fed into a GRU cell, and the GRU state was used to estimate the desired thrust acceleration and current velocity through a fully connected layer. Supplementary Fig. 5 illustrates the data flow in our RNN unfolded in time.

Hardware set-up

Our hardware set-up is visualized in Fig. 2a. The hardware developed in this work was designed to be small, agile and fully autonomous while remaining low cost. We set up our quadrotor as a first-person-view drone. It had a total weight of only 365 g. The aircraft was built using a Roma 3-inch frame, GEMFAN 3-inch propellers, and 1606 3750KV motors. The maximum thrust-to-weight ratio was 3.6, thereby enabling the agility of our quadrotor. We used a commonly adopted flight tower for the first-person view, which consisted of an Aocoda F7mini flight controller and a HAKRC 4-in-1 electronic speed controller. Furthermore, we used a customized version of the firmware BetaFlight (<https://betaflight.com>) for autonomous flights. Regarding the perception system, a RealSense D435i camera was used as a depth-perception module, which output depth images at a rate of 15 Hz. Additionally, we used a compact single-board computer, Mango Pi (<https://mangopi.org/mqquad>), as our onboard computer. It measures $30 \times 65 \times 6$ mm³, weighs just 11 g and is equipped with a Cortex-A53 CPU. We integrated the computer into the middle of the aircraft carbon boards and combined it with the flight controller to form a three-layer flight tower. Thanks to the extremely lightweight network design, our whole system can run in real time with such a compact structure.

Related works

Traditional approaches^{28,47,48} for vision-based navigation are largely mapping-based. They first build a high-quality map using occupancy grids⁴⁸ or the Euclidean signed distance field²⁸ as a representation of the three-dimensional environment. Then, the navigation problem is converted into planning and control. Planning typically relies on probabilistic search or gradient-based trajectory optimization, whereas control often uses model-based techniques, such as model predictive control^{49,50}. To enhance safety and computational efficiency, certain strategies integrate a safe flight corridor as an intermediate representation of the obstacles for planning^{51–53}. The modular design allows individual optimization of components for better performance. However, dividing the navigation task into mapping, planning and control results in pipelines that may overlook the interplay between different stages, leading to compounded errors. For example, state estimation during high-speed flight is prone to drift^{22,54,55}, which causes deviations from the planned trajectories and shifts in the map fusion. Mapping errors may further cause suboptimal trajectory planning. Furthermore, the sequential nature of these pipelines introduces extra latency and computational burden, thereby hindering their feasibility in complex or dynamic environments.

Unlike mapping-based methods, an end-to-end policy learned for vision-based navigation can directly control the vehicle from raw sensory observations without explicitly mapping and planning. In particular, imitation learning is an effective approach for training a deep sensorimotor policy, in which the policy is trained by imitating either a demonstration by human experts or trajectory roll-outs by a model-based controller. Imitation learning has been shown to be effective in solving several challenging tasks, including vision-based drone acrobatics¹⁵, navigation in the wild^{4,8,56} and drone racing^{57–61}. Although effective for specific tasks, imitation learning heavily depends on task-specific expert demonstrations, limiting its adaptability to new scenarios. Recently, Xing et al.⁶² used state-based reinforcement learning to build an expert model for drone racing, but its state space was designed specifically for racing and is unsuitable for obstacle avoidance in unstructured environments. Bhattacharya et al.⁶³ explored vision transformers for obstacle avoidance by drones, but their approach still relies on expert methods and odometry and they did not explore flights beyond 7 m s⁻¹. Their method requires a laptop-grade processor, whereas our system employs a lightweight neural network that runs on a US\$21 low-cost computer.

A differentiable-physics simulation is a powerful technique that applies gradient-based methods to the learning and control of physical systems⁶⁴. Previous studies have demonstrated the efficiency of differentiable programming approaches in various robotics simulation tasks, such as control of soft robots^{65–67}, mimicking human motion by humanoid robots⁶⁸, system identification⁶⁹ and quadruped locomotion^{70,71}. However, these works do not involve obstacle avoidance with vision inputs. Differentiable programming has challenges, most notably its susceptibility to gradient explosions. Consequently, developing a practical differentiable simulation framework for tackling real-world quadrotor navigation tasks remains an open research challenge. Notably, existing autonomous quadrotor applications have yet to adopt differentiable simulation-based methods for real-world navigation.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

The videos of flight recordings of the real-world experiments and collision and flight time data in simulated experiments are available on figshare at <https://doi.org/10.6084/m9.figshare.26298379> (ref. 72).

Code availability

The code for single- and multi-agent training and simulation experiments is available on Zenodo at <https://doi.org/10.5281/zenodo.15250256> (ref. 73).

References

- Schedl, D. C., Kurmi, I. & Bimber, O. An autonomous drone for search and rescue in forests using airborne optical sectioning. *Sci. Robot.* **6**, 1188 (2021).
- Xing, J., Cioffi, G., Hidalgo-Carrió, J. & Scaramuzza, D. Autonomous power line inspection with drones via perception-aware MPC. In *Proc. 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 1086–1093 (IEEE, 2023).
- Sage, A. T. et al. Testing the delivery of human organ transportation with drones in the real world. *Sci. Robot.* **7**, 5798 (2022).
- Giusti, A. et al. A machine learning approach to visual perception of forest trails for mobile robots. *IEEE Robot. Autom. Lett.* **1**, 661–667 (2015).
- Gao, F. et al. Teach-repeat-replan: a complete and robust system for aggressive flight in complex environments. *IEEE Trans. Robot.* **36**, 1526–1545 (2020).
- Zhou, X., Wang, Z., Ye, H., Xu, C. & Gao, F. Ego-planner: an ESDF-free gradient-based local planner for quadrotors. *IEEE Robot. Autom. Lett.* **6**, 478–485 (2020).
- Zhou, X. et al. Swarm of micro flying robots in the wild. *Sci. Robot.* **7**, 5954 (2022).
- Loquercio, A. et al. Learning high-speed flight in the wild. *Sci. Robot.* **6**, 5810 (2021).
- Zhang, Z. & Scaramuzza, D. Perception-aware receding horizon navigation for MAVs. In *Proc. 2018 IEEE International Conference on Robotics and Automation (ICRA)* 2534–2541 (IEEE, 2018).
- Maimone, M. W., Leger, P. C. & Biesiadecki, J. J. Overview of the Mars Exploration Rovers' autonomous mobility and vision capabilities. In *Proc. IEEE International Conference on Robotics and Automation (ICRA) Space Robotics Workshop* (IEEE, 2007).
- Hwangbo, J. et al. Learning agile and dynamic motor skills for legged robots. *Sci. Robot.* **4**, 5872 (2019).
- Miki, T. et al. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Sci. Robot.* **7**, 2822 (2022).
- Choi, S. et al. Learning quadrupedal locomotion on deformable terrain. *Sci. Robot.* **8**, 2256 (2023).
- Song, Y., Romero, A., Mueller, M., Koltun, V. & Scaramuzza, D. Reaching the limit in autonomous racing: optimal control versus reinforcement learning. *Sci. Robot.* <https://doi.org/10.1126/scirobotics.adg1462> (2023).
- Kaufmann, E. et al. Deep drone acrobatics. In *Proc. Robotics: Science and Systems* (eds Toussaint, M. et al.) (RSS Foundation, 2020).
- Song, Y., Shi, K., Penicka, R. & Scaramuzza, D. Learning perception-aware agile flight in cluttered environments. In *Proc. 2023 IEEE International Conference on Robotics and Automation (ICRA)* 1989–1995 (IEEE, 2023).
- Sadeghi, F. & Levine, S. CAD2RL: real single-image flight without a single real image. In *Proc. Robotics: Science and Systems XIII* (eds Amato, A. et al.) (RSS Foundation, 2017).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. Preprint at <https://arxiv.org/abs/1707.06347> (2017).
- Foehn, P. et al. Agilicious: open-source and open-hardware agile quadrotor for vision-based flight. *Sci. Robot.* **7**, 6259 (2022).
- Shahzad, M. M. et al. A review of swarm robotics in a nutshell. *Drones* **7**, 269 (2023).
- Kegeleirs, M., Grisetti, G. & Birattari, M. Swarm SLAM: challenges and perspectives. *Front. Robot. AI* **8**, 618268 (2021).

22. Delmerico, J., Cieslewski, T., Rebecq, H., Faessler, M. & Scaramuzza, D. Are we ready for autonomous drone racing? The UZH-FPV drone racing dataset. In *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 6713–6719 (IEEE, 2019).
23. Cioffi, G., Bauersfeld, L., Kaufmann, E. & Scaramuzza, D. Learned inertial odometry for autonomous drone racing. *IEEE Robot. Autom. Lett.* **8**, 2684–2691 (2023).
24. Kaufmann, E. et al. Champion-level drone racing using deep reinforcement learning. *Nature* **620**, 982–987 (2023).
25. Qin, T., Li, P. & Shen, S. VINS-Mono: a robust and versatile monocular visual-inertial state estimator. *IEEE Trans. Robot.* **34**, 1004–1020 (2018).
26. Selvaraju, R. R. et al. Grad-cam: visual explanations from deep networks via gradient-based localization. In *Proc. IEEE International Conference on Computer Vision* (eds Cucchiara, R. et al.) 618–626 (IEEE, 2017).
27. Song, Y., Naji, S., Kaufmann, E., Loquercio, A. & Scaramuzza, D. Flightmare: a flexible quadrotor simulator. In *Proc. Conference on Robot Learning* (eds Faust, A. et al.) 1147–1157 (PMLR, 2021).
28. Zhou, B., Gao, F., Wang, L., Liu, C. & Shen, S. Robust and efficient quadrotor trajectory generation for fast autonomous flight. *IEEE Robot. Autom. Lett.* **4**, 3529–3536 (2019).
29. Florence, P., Carter, J. & Tedrake, R. Integrated perception and control at high speed: evaluating collision avoidance maneuvers without maps. In *Proc. 12th Workshop on the Algorithmic Foundations of Robotics* (eds Goldberg, K. et al.) 304–319 (Springer, 2020).
30. Shah, S., Dey, D., Lovett, C. & Kapoor, A. in *Field and Service Robotics* (eds Hutter, M. & Siegart, R.) Ch. 40 (Springer, 2017).
31. Ross, S., Gordon, G. & Bagnell, D. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proc. 14th International Conference on Artificial Intelligence and Statistics* (eds Gordon, G. et al.) 627–635 (PMLR, 2011).
32. Gurumurthy, S., Kolter, J. Z. & Manchester, Z. Deep off-policy iterative learning control. In *Proc. 5th Annual Learning for Dynamics & Control Conference* (eds Matni, N. et al.) 639–652 (PMLR, 2023).
33. Cho, K. et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proc. 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (eds Moschitti, A. et al.) 1724–1734 (Association for Computational Linguistics, 2014).
34. Suh, H. J., Simchowit, M., Zhang, K. & Tedrake, R. Do differentiable simulators give better policy gradients? In *Proc. International Conference on Machine Learning* (eds Chaudhuri, K. et al.) 20668–20696 (PMLR, 2022).
35. Haarnoja, T., Zhou, A., Abbeel, P. & Levine, S. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proc. International Conference on Machine Learning* (eds Dy, J. et al.) 1861–1870 (PMLR, 2018).
36. Metz, L., Freeman, C. D., Schoenholz, S. S. & Kachman, T. Gradients are not all you need. Preprint at <https://arxiv.org/abs/2111.05803> (2021).
37. Paszke, A. et al. PyTorch: an imperative style, high-performance deep learning library. Preprint at <https://arxiv.org/abs/1912.01703> (2019).
38. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Comput.* **9**, 1735–1780 (1997).
39. O’Connell, M. et al. Neural-fly enables rapid learning for agile flight in strong winds. *Sci. Robot.* **7**, 6597 (2022).
40. Todorov, E., Erez, T. & Tassa, Y. Mujoco: a physics engine for model-based control. In *Proc. 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems* (eds Guglielmelli, G. et al.) 5026–5033 (IEEE, 2012).
41. Faessler, M., Franchi, A. & Scaramuzza, D. Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories. *IEEE Robot. Autom. Lett.* **3**, 620–626 (2018).
42. Hu, Y. et al. Seeing through pixel motion: learning obstacle avoidance from optical flow with one camera. *IEEE Robot. Autom. Lett.* **10**, 5871–5878 (2024).
43. Girshick, R. Fast R-CNN. In *Proc. 2015 IEEE International Conference on Computer Vision (ICCV)* (eds Ikeuchi, K. et al.) 1440–1448 (IEEE, 2015).
44. Dugas, C., Bengio, Y., Bélisle, F., Nadeau, C. & Garcia, R. Incorporating second-order functional knowledge for better option pricing. In *Proc. 13th Conference on Neural Information Processing Systems* (eds Leen, T. et al.) 451–457 (MIT Press, 2000).
45. Bengio, Y., Simard, P. & Frasconi, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Trans. Neural Netw.* **5**, 157–166 (1994).
46. Shi, B., Bai, X. & Yao, C. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* **39**, 2298–2304 (2016).
47. Scaramuzza, D. et al. Vision-controlled micro flying robots: from system design to autonomous navigation and mapping in GPS-denied environments. *IEEE Robot. Autom. Mag.* **21**, 26–40 (2014).
48. Liu, S., Mohta, K., Atanasov, N. & Kumar, V. Search-based motion planning for aggressive flight in SE(3). *IEEE Robot. Autom. Lett.* **3**, 2439–2446 (2018).
49. Neunert, M. et al. Fast nonlinear model predictive control for unified trajectory optimization and tracking. In *Proc. 2016 IEEE International Conference on Robotics and Automation (ICRA)* (eds Bicchi, A. & De Luca, A.) 1398–1404 (IEEE, 2016).
50. Falanga, D., Foehn, P., Lu, P. & Scaramuzza, D. Pampc: perception-aware model predictive control for quadrotors. In *Proc. 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (ed. Maciejewski, A. A.) 1–8 (IEEE, 2018).
51. Ji, J., Wang, Z., Wang, Y., Xu, C. & Gao, F. Mapless-planner: a robust and fast planning framework for aggressive autonomous flight without map fusion. In *Proc. 2021 IEEE International Conference on Robotics and Automation (ICRA)* 6315–6321 (IEEE, 2021).
52. Gao, F., Wu, W., Gao, W. & Shen, S. Flying on point clouds: online trajectory generation and autonomous navigation for quadrotors in cluttered environments. *J. Field Robot.* **36**, 710–733 (2018).
53. Tordesillas, J. & How, J. P. Mader: trajectory planner in multiagent and dynamic environments. *IEEE Trans. Robot.* **38**, 463–476 (2021).
54. Wang, W. et al. Tartanair: a dataset to push the limits of visual SLAM. In *Proc. 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 4909–4916 (IEEE, 2020).
55. Teed, Z. & Deng, J. DROID-SLAM: deep visual SLAM for monocular, stereo, and RGB-D cameras. In *Proc. 34th Conference on Advances in Neural Information Processing Systems* (eds Ranzato, M. et al.) 16558–16569 (Curran Associates, 2021).
56. Vorbach, C., Hasani, R., Amini, A., Lechner, M. & Rus, D. Causal navigation by continuous-time neural networks. In *Proc. 34th Conference on Advances in Neural Information Processing Systems* (eds Ranzato, M. et al.) 12425–12440 (Curran Associates, 2021).
57. Li, G. et al. Oil: observational imitation learning. Preprint at <https://arxiv.org/abs/1803.01129> (2019).
58. Kaufmann, E. et al. Deep drone racing: learning agile flight in dynamic environments. In *Proc. Conference on Robot Learning* (eds Billard, A. et al.) 133–145 (PMLR, 2018).
59. Loquercio, A. et al. Deep drone racing: from simulation to reality with domain randomization. *IEEE Trans. Robot.* **36**, 1–14 (2019).

60. Wang, T. & Chang, D. E. Robust navigation for racing drones based on imitation learning and modularization. In *Proc. 2021 IEEE International Conference on Robotics and Automation (ICRA)* 13724–13730 (IEEE, 2021).
61. Fu, J., Song, Y., Wu, Y., Yu, F. & Scaramuzza, D. Learning deep sensorimotor policies for vision-based autonomous drone racing. In *Proc. 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 5243–5250 (IEEE, 2023).
62. Xing, J., Romero, A., Bauersfeld, L. & Scaramuzza, D. Bootstrapping reinforcement learning with imitation for vision-based agile flight. In *Proc. 8th Annual Conference on Robot Learning* (2024).
63. Bhattacharya, A. et al. Vision transformers for end-to-end vision-based quadrotor obstacle avoidance. In *Proc. 2025 IEEE International Conference on Robotics and Automation (ICRA)* (IEEE, 2025).
64. Liang, J. & Lin, M. C. Differentiable physics simulation. In *Proc. ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations* (eds Mohamed, S. et al.) (2020).
65. Hu, Y. et al. DiffTaichi: differentiable programming for physical simulation. In *Proc. International Conference on Learning Representations* (2020).
66. Hu, Y. et al. Chainqueen: a real-time differentiable physical simulator for soft robotics. In *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 6265–6271 (IEEE, 2019).
67. Bern, J. M., Schneider, Y., Banzet, P., Kumar, N. & Coros, S. Soft robot control with a learned differentiable model. In *Proc. 2020 3rd IEEE International Conference on Soft Robotics (RoboSoft)* (ed. Tolley, M.) 417–423 (IEEE, 2020).
68. Ren, J. et al. Diffmimic: efficient motion mimicking with differentiable physics. In *Proc. 11th International Conference on Learning Representations* (2023).
69. Jatavallabhula, K. M. et al. gradsim: differentiable simulation for system identification and visuomotor control. In *Proc. International Conference on Learning Representations (ICLR)* (eds Oh, A. et al.) (2021).
70. Song, Y., Kim, S. & Scaramuzza, D. Learning quadruped locomotion using differentiable simulation. In *Proc. 8th Annual Conference on Robot Learning* (2024).
71. Schwarke, C., Klemm, V., Tordesillas, J., Sleiman, J.-P. & Hutter, M. Learning quadrupedal locomotion via differentiable simulation. Preprint at <https://arxiv.org/abs/2404.02887> (2024).
72. Zhang, Y. & Hu, Y. Data for learning vision-based agile flight via differentiable physics. *figshare* <https://doi.org/10.6084/m9.figshare.26298379> (2025).
73. Zhang, Y. & Hu, Y. Code for learning vision-based agile flight via differentiable physics. *Zenodo* <https://doi.org/10.5281/zenodo.15250256> (2025).

Acknowledgements

This project is supported by the National Natural Science Foundation of China (Grant Nos 62325109 and U21B2013 to W.L. and Grant No. 62073214 to D.Z.). We thank SJTU SEIEE · G60 Yun Zhi AI Innovation and Application Research Center for indoor experiment support, J. Li for initial efforts in the multi-agent experiments, and L. Zhang and F. Yu for helping with the experiments and the valuable discussions.

Author contributions

Y.Z. formulated the main ideas, implemented the system, performed the experiments, analysed the data and wrote the paper. Y.H. implemented the system, performed the experiments, produced the data visualization and wrote the paper. Y.S. contributed to project conception, data visualization and writing the paper. D.Z. contributed to the project conception and revised the paper. W.L. directed the research, contributed to the paper writing and provided funding.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s42256-025-01048-0>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s42256-025-01048-0>.

Correspondence and requests for materials should be addressed to Danping Zou or Weiyao Lin.

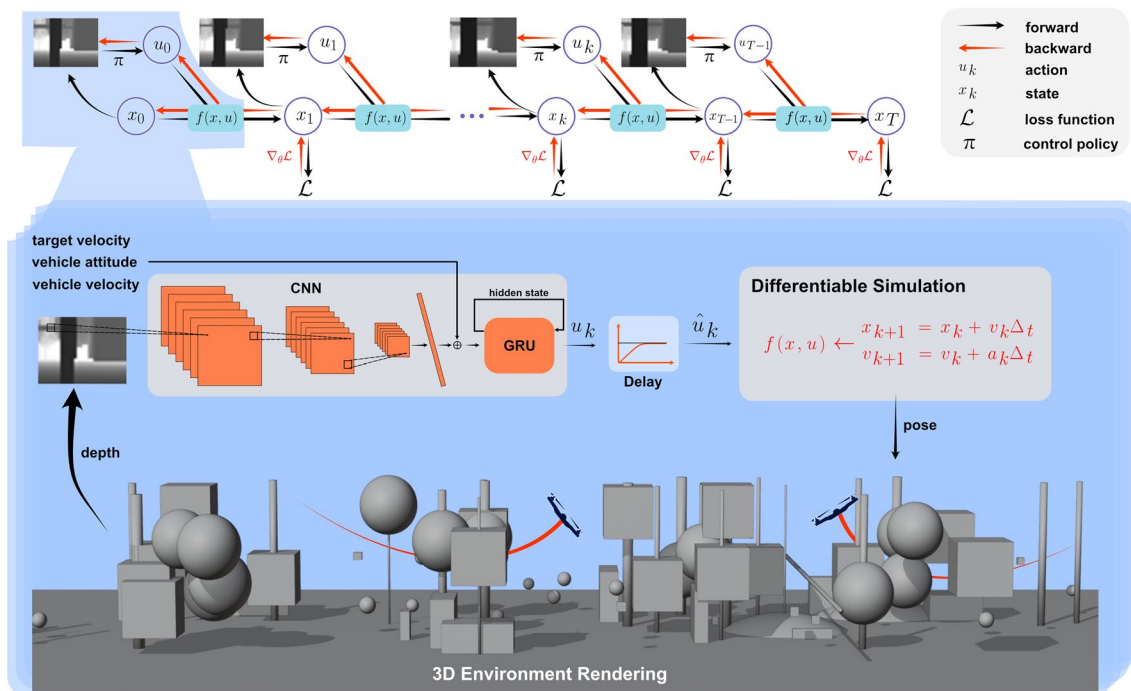
Peer review information *Nature Machine Intelligence* thanks Pratik Kunapuli and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

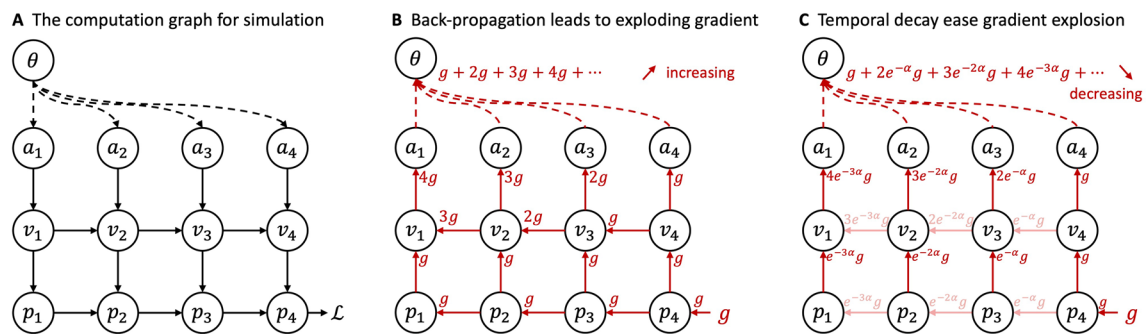
Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2025



Extended Data Fig. 1 | Method overview: learning vision-based agile flight via differentiable physics. The differentiable simulator carries gradients from the output state directly to the control inputs, and hence, to the policy parameters.

A single timestep consists of depth map rendering, action prediction, and quadrotor dynamics simulation. The training environment only includes randomly placed obstacles.



Extended Data Fig. 2 | A computation graph of the physics simulation. Temporal gradient decay mitigates gradient explosion. We illustrate the simplified model $v_t = v_{t-1} + a_t$, $p_t = p_{t-1} + v_t$.

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our [Editorial Policies](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a Confirmed

- ☒ ☐ The exact sample size (n) for each experimental group/condition, given as a discrete number and unit of measurement
- ☐ ☒ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
- ☒ ☐ The statistical test(s) used AND whether they are one- or two-sided
Only common tests should be described solely by name; describe more complex techniques in the Methods section.
- ☐ ☒ A description of all covariates tested
- ☒ ☐ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
- ☐ ☒ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
- ☒ ☐ For null hypothesis testing, the test statistic (e.g. F , t , r) with confidence intervals, effect sizes, degrees of freedom and P value noted
Give P values as exact values whenever suitable.
- ☒ ☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
- ☐ ☒ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
- ☒ ☐ Estimates of effect sizes (e.g. Cohen's d , Pearson's r), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection	All collected data are available on Figshare at https://figshare.com/s/ca10b90e364f88e537fa . We use PyTorch 2.2.2 and CUDA 11.8 for training. For our testing of Agile Autonomy (Fig. 6a) and the forest simulation environment (Fig. 5a), we use https://github.com/uzh-rpg/agile_autonomy at commit '958c0d2' and AirSim version 1.8.1.
Data analysis	The code for data analysis, along with the associated datasets, is publicly accessible on Figshare at https://doi.org/10.6084/m9.figshare.26298379 . The code for training and simulation experiments is available at https://sjtueducn-my.sharepoint.com/:f:/g/personal/zyayoung_sjtu_edu_cn/EjDy1kFC585No8ielcTyMcABITxtYFJFi7WQbubki6t9sA?e=G5KO6E and will be accessible on GitHub upon publication.

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our [policy](#)

The recorded collision and flight time data in simulated experiments, as well as flight recording videos for real-world experiments, are available in Figshare at <https://doi.org/10.6084/m9.figshare.26298379>.

Human research participants

Policy information about [studies involving human research participants and Sex and Gender in Research](#).

Reporting on sex and gender

n/a

Population characteristics

n/a

Recruitment

n/a

Ethics oversight

n/a

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☒ Life sciences ☐ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see [nature.com/documents/nr-reporting-summary-flat.pdf](https://www.nature.com/documents/nr-reporting-summary-flat.pdf)

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size

No sample size calculation was performed. The chosen sample sizes, i.e., 10 trials per speed per environment following practices in Loquercio et al. [1]. Training ablation experiments are repeated with 5 different random seeds following Xu et al. [2].

[1] Loquercio, Antonio, et al. "Learning high-speed flight in the wild." Science Robotics 6.59 (2021): eabg5810.

[2] Xu, Jie, et al. "Accelerated policy learning with parallel differentiable simulation." International Conference on Learning Representations (2022).

Data exclusions

No data were excluded.

Replication

All experiments are reproducible with released data and code. All experiments were replicated across multiple trials (e.g., 10 trials per condition) and/or environments to verify reproducibility.

Randomization

Sample groups and covariates (e.g., training reward, speed) are controlled by designed test conditions, including different training seeds and randomized environments or initial positions.

Blinding

Not relevant. The experiments involved autonomous systems and objective performance metrics (e.g., success rate, speed) that do not require subjective interpretation.

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern

Methods

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging